AppBana Genesis

Comprehensive Architecture Document

AI-native enterprise application creation platform

Draft v1.0 | North Star Architecture + Phase 0 Implementation Foundation | July 2026

North Star: turn business intent into governed, technology-independent enterprise applications.

Business idea -> AI Application Agent -> Canonical Application Model -> Runtime Engines -> Technology Adapters -> Enterprise Application

Document Map

This document is the revised umbrella architecture for AppBana Genesis. It supersedes the narrower "metadata-driven UI runtime" framing by positioning the previous interaction runtime as one subsystem inside a broader business-intent-driven enterprise application platform.

- Executive architecture summary and product thesis
-
- North Star architecture and revised diagram
- Architecture principles and scope boundaries
- Business Intent Model and AI Application Agent
- Governance, validation, canonical application model, and platform kernel
- Runtime engines: UI, workflow, rules, API, data, integration, security, observability
- Technology adapter layer and replaceability model
- Control planes: registry, migration, conformance, AI governance, risk, traceability
- Customer Onboarding as first vertical application slice
- Implementation modules, repo structure, roadmap, decision gates, and open questions

1. Executive Architecture Summary

AppBana Genesis is an AI-native enterprise application creation platform. The platform converts business intent into governed application metadata, normalizes that metadata into a canonical application model, executes the model through technology-independent runtime engines, and deploys or renders the application through replaceable technology adapters.

This architecture is intentionally broader than a metadata-driven UI renderer. UI rendering is one runtime engine; the full platform covers UI, workflow, rules, data, APIs, security, integrations, observability, and deployment.

Dimension	Architecture Position
Product category	AI-native enterprise application creation platform.
Core promise	Business intent survives technology change.
Primary user	Business/product user assisted by AI and governed by enterprise architecture controls.
Core asset	Canonical Application Model plus governance, runtime engines, conformance, and technology adapters.
First vertical slice	Customer Onboarding Application.
Not a goal	Replacing all frontend/backend development with unrestricted metadata or AI-generated code.

2. Revised North Star Architecture

The following diagram shows AppBana Genesis as the umbrella platform. The previously defined interaction/runtime architecture remains important, but it is now one subsystem in the broader product.

AppBana Genesis - Revised North Star Architecture

AI-native enterprise application creation platform: business intent to governed, technology-independent applications

North Star: Business intent survives technology change

Business User / Product Owner

Business idea, goals, policies, constraints

AI Application Agent

Conversation, clarification, decomposition, patch proposal

Business Intent Model

Capability, process, entities, rules, personas, outcomes

Governance + Validation Plane

Policy, security, privacy, accessibility, approval gates

Canonical Application Model

UI + workflow + data + APIs + security + integrations

AppBana Platform Kernel

Deterministic execution, versioning, migration, trace, explainability

Runtime Engines

Technology-independent execution engines generated from the canonical application model

- **Interaction / UI Runtime** — Screens, components, render projection
- **Workflow Runtime** — States, tasks, approvals, transitions
- **Rules Runtime** — Business rules, validation, decisions
- **API / Operation Runtime** — Semantic operations, contracts, effects
- **Data Runtime** — Entities, persistence, query model
- **Integration Runtime** — Events, connectors, external systems
- **Security / Policy Runtime** — Roles, access, data classification
- **Observability Runtime** — Trace, audit, metrics, explanations

Technology Adapter Layer

Replaceable implementation adapters keep business applications insulated from technology churn

- **UI Adapters** — React, Web Components, future UI tech
- **Backend Adapters** — Java, .NET, Node, Python, future stacks
- **Data Adapters** — SQL, NoSQL, object storage, vector stores
- **Integration Adapters** — REST, GraphQL, events, queues, connectors
- **Cloud / Deployment** — Azure, AWS, GCP, on-prem, hybrid
- **Identity / Enterprise** — IAM, policy engines, audit, compliance

Cross-Cutting Control Planes

- **Metadata Registry** — Immutable artifacts, versions, rollout, rollback
- **Conformance Suite** — Runtime and adapter certification tests
- **Migration Engine** — Schema evolution, reports, compatibility
- **AI Governance** — Patch provenance, diff, validation, approval
- **Risk & Policy** — Security, privacy, accessibility, compliance
- **Traceability** — Explain what happened, why, and under which artifact

Generated Enterprise Applications

Business-user-driven applications with UI, workflow, data, APIs, security, integration, audit, and deployment.

First vertical slice: Customer Onboarding Application

Architecture principle: AppBana Genesis is the umbrella application creation platform. The existing UI/interaction runtime is one subsystem, not the whole product.

Figure 1. AppBana Genesis revised north star architecture.

3. Product Vision and Architecture Thesis

The long-term vision is that a business user can express a business idea in natural language and guided conversation. An AI Application Agent translates that idea into an enterprise application intent model. AppBana Genesis validates, governs, generates, executes, and evolves that application without binding the business intent to a specific UI framework, backend language, database, cloud provider, or integration technology.

Business Goal	Architecture Response
Business users should describe outcomes rather than frameworks.	AI Application Agent captures business intent and converts it to governed application intent.
Technology should be replaceable.	Technology adapters isolate UI, backend, data, integration, cloud, and identity implementations.
Enterprise applications require governance.	Validation and policy planes enforce security, privacy, accessibility, compatibility, and approval gates.
AI must be safe.	AI proposes patches and application intent; publication requires validation, diff, preview, and approval.
Applications must be explainable.	Runtime traces and artifact versions explain what happened, why, and under which rule/version.

4. Core Architecture Principles

Principle	Meaning	Implementation Implication
Business intent first	Business capability, process, rules, roles, and outcomes are primary.	Business Intent Model sits above technical metadata.
Canonical application model	Runtimes and adapters consume normalized model, not raw AI output or raw metadata.	Migration/normalization pipeline is mandatory.
Runtime engines over code generation	Generated code may exist later, but runtime semantics are primary.	Behavior is deterministic, traceable, governed.
Technology adapters are replaceable	Adapters implement technology choices without changing business intent.	UI/backend/data/cloud adapters require contracts and conformance.
No arbitrary executable metadata	Metadata cannot become a hidden programming language.	Use declarative models, semantic operations, and registered extensions.
Governance is product foundation	Validation, policy, approval, audit, and rollback are not optional.	Publication gate is part of the core platform.
AI is an agent, not an unchecked deployer	AI assists, proposes, explains, and migrates, but cannot silently publish.	AI governance and provenance are required.
Backend/domain remains authoritative	UI and workflow improve interaction; server/domain policies enforce truth.	Semantic operation contracts enforce domain rules.

5. Architecture Drivers and Non-Goals

Driver	Why It Matters

Technology churn	Frameworks, languages, databases, and cloud patterns will keep changing.
Enterprise complexity	Real applications require UI, workflow, data, API, security, integrations, audit, and lifecycle management.
AI acceleration	LLMs can help translate and maintain intent, but only when constrained by schema, validation, and governance.
Compliance and audit	Enterprise platforms must reconstruct what users saw, which rules fired, and why decisions happened.
Productization	The platform must avoid becoming a bespoke services factory.

- Non-goal: unrestricted AI code generation into production applications.
- Non-goal: replacing every bespoke visual or highly interactive application pattern.
- Non-goal: making metadata a general-purpose programming language.
- Non-goal: generating only CRUD forms and calling it enterprise application creation.

6. Business Intent Model and AI Application Agent

The Business Intent Model is the top-level representation of what the business wants to achieve. The AI Application Agent is responsible for eliciting, clarifying, decomposing, and proposing application intent artifacts, but not for directly publishing production behavior.

Business Intent Category	Examples
Business capability	Customer onboarding, claim intake, employee transfer, supplier registration.
Personas and roles	Applicant, reviewer, manager, auditor, administrator.
Process and workflow	Draft, submit, review, approve, reject, request more info.
Business entities	Customer, onboarding case, document, risk assessment, approval decision.
Rules and policies	Tax id required by country, approval required for high risk, field visible by role.
Data and retention	PII classification, document retention, audit retention, redaction rules.
Operational constraints	SLA, approval hierarchy, localization, integration needs.
AI Agent Responsibility	**Control Required**
Clarify requirements	Ask missing questions before artifact proposal.
Generate application intent	Produce structured intent model, not raw app code.
Propose changes	Generate patch, not uncontrolled full rewrite.
Explain behavior	Use trace/model/rules to explain application behavior.
Suggest migrations	Suggest schema/model migration but require deterministic tests.
Publish	Not allowed directly; requires governance workflow.

7. Governance and Validation Plane

Governance is a first-class architecture layer between business/AI intent and executable application runtime. The validation plane prevents unsafe, incompatible, inaccessible, non-compliant, or non-performant applications from being activated.

Validation Type	Purpose
Intent validation	Does the business intent contain enough information to produce an application model?
Schema validation	Does the application metadata conform to versioned schema?
Security validation	Are roles, permissions, field exposure, and operations safe?
Privacy validation	Are PII and sensitive data classified and redacted where required?
Accessibility validation	Can generated UI satisfy baseline accessibility expectations?
Compatibility validation	Do runtime engines and technology adapters support required capabilities?
Operation validation	Are all operations registered, versioned, authorized, idempotency-safe where needed?
Performance validation	Are components, rules, operations, metadata size, and dependencies within budget?
AI governance validation	Does an AI-generated patch include provenance, diff, preview, and approval path?

8. Canonical Application Model

The Canonical Application Model is the central normalized representation of an enterprise application. It is broader than a canonical UI model and contains multiple sub-models that are consumed by runtime engines and technology adapters.

Sub-Model	Purpose	Examples
Application Identity Model	Application metadata, version, owner, lifecycle.	appId, artifactVersion, status, rollout pointer.
Domain/Data Model	Entities, fields, relationships, classifications.	Customer, Document, OnboardingCase.
Interaction/UI Model	Screens, forms, components, render projection.	Onboarding form, review screen, audit view.
Workflow Model	States, transitions, tasks, approvals.	draft -> submitted -> reviewed -> approved.
Rule Model	Business rules, validations, derived conditions.	GST required for Indian business customers.
Operation/API Model	Semantic operations and contracts.	customer.saveDraft, document.upload.
Security/Policy Model	Roles, permissions, field policies, data access.	Applicant can edit own draft.
Integration Model	External systems, events, connectors.	Tax validation service, document storage.

Observability Model	Trace, audit, telemetry, explanation.	Why field visible, which rule fired.
Deployment Model	Target environments and adapter requirements.	React UI, Java service, SQL persistence, Azure deployment.

9. AppBana Platform Kernel

The platform kernel is the deterministic execution and lifecycle core. It coordinates model resolution, versioning, runtime sessions, rule graphs, operation planning, migration, trace, and governance.

Kernel Capability	Responsibility
Artifact resolution	Resolve active application version by tenant, environment, user, region, feature flag, and rollout policy.
Migration and normalization	Convert older metadata/application intent into canonical model.
Runtime session management	Initialize runtime engines and share context across UI, workflow, data, rules, and operations.
Event orchestration	Route events between runtime engines while preserving deterministic trace.
Effect planning	Convert actions/workflow transitions/operations into controlled effect descriptors.
Trace and explainability	Capture why application behavior occurred under exact artifact versions.
Policy enforcement	Apply governance gates before activation and critical runtime actions.
Adapter negotiation	Confirm technology adapters can support required capabilities.

10. Runtime Engines

Runtime engines execute different parts of the canonical application model. Each engine has a contract, lifecycle, trace model, and capability set. Engines should be independently replaceable or upgradeable over time.

Runtime Engine	Scope	Initial Phase 0/1 Priority
Interaction / UI Runtime	Screens, components, forms, render projection, UI events.	High: first implemented runtime.
Workflow Runtime	States, tasks, approvals, transitions, lifecycle.	Medium: model in pilot, fuller engine later.
Rules Runtime	Business conditions, validations, derived state, decision support.	High: required for dynamic behavior.
API / Operation Runtime	Semantic operations, effect execution, error taxonomy.	High: save, validate, upload, submit.
Data Runtime	Entities, persistence strategy, query/read models.	Medium: mock/persistence abstraction initially.
Integration Runtime	External connectors, events, callbacks, async integrations.	Low-medium: tax validation/document storage mock first.

Security / Policy Runtime	RBAC/ABAC, field policy, operation auth, data classification.	High: role-based pilot behavior.
Observability Runtime	Trace, audit, metrics, explanation, replay.	High: key differentiator.

11. Technology Adapter Layer

Technology adapters translate runtime capabilities into concrete implementation technologies. This is where React, Web Components, Java, .NET, SQL, NoSQL, REST, events, cloud providers, and identity products are connected without changing the business intent model.

Adapter Category	Examples	Replaceability Strategy
UI adapters	React, Web Components, future UI frameworks.	Conformance tests for Tier A/B UI capabilities.
Backend adapters	Java, .NET, Node, Python, future stacks.	Semantic operation contracts isolate business operations.
Data adapters	SQL, NoSQL, object storage, document storage, vector stores.	Canonical data model maps to storage-specific generated or managed artifacts.
Integration adapters	REST, GraphQL, event bus, queues, enterprise connectors.	Integration contracts and error taxonomy.
Cloud/deployment adapters	Azure, AWS, GCP, on-prem, hybrid.	Deployment model and infrastructure policy abstractions.
Identity adapters	OIDC, SAML, enterprise IAM, policy engines.	Security/policy runtime uses normalized identity context.

12. Cross-Cutting Control Planes

Control Plane	Purpose	Why It Matters
Metadata/Application Registry	Stores immutable artifacts, versions, rollout, rollback, provenance.	Supports audit, reproducibility, safe release.
Migration Engine	Evolves schemas/models while preserving business intent.	Prevents technology and schema churn from breaking apps.
Conformance Suite	Certifies runtime engines and adapters against contracts.	Prevents adapter drift and false portability.
AI Governance	Controls prompt/context, patch provenance, validation, diff, approval.	Keeps AI safe and explainable.
Risk and Policy	Security, privacy, accessibility, compliance, performance controls.	Makes generated apps enterprise-ready.
Traceability and Observability	Explains what happened, why, and under which artifact.	Supports support, audit, compliance, debugging.

13. Application Lifecycle

Lifecycle Stage	Description	Primary Artifacts

Idea capture	Business user describes desired application outcome.	Conversation transcript, initial intent.
Clarification	AI agent asks missing questions and identifies constraints.	Clarification log, assumptions.
Intent modeling	Business intent becomes structured application intent model.	Business Intent Model.
Governance validation	Intent is checked for completeness, policy, security, and feasibility.	Validation report.
Canonical generation	Application intent is normalized into Canonical Application Model.	Canonical application artifact.
Runtime activation	Runtime engines initialize application behavior.	Runtime session, trace context.
Adapter realization	Technology adapters render/deploy UI, operations, data, integrations.	Adapter artifacts.
Operation and monitoring	Application runs with trace, audit, metrics, and explanation.	Trace, telemetry, audit.
Change and migration	AI/user proposes changes; system validates, diffs, migrates, publishes.	Patch, diff, migration report, version.

14. Security, Privacy, and Compliance Architecture

Because AppBana Genesis can create full enterprise applications, security and compliance must be part of the creation path, not added after generation.

Security Concern	Architecture Control
Unauthorized field exposure	Field-level policy and projection filtering.
Unsafe operations	Only registered semantic operations with authorization contracts.
PII leakage	Data classification, telemetry redaction, privacy validation.
AI unsafe changes	Patch workflow with provenance, validation, diff, preview, approval.
Artifact tampering	Immutable artifacts, checksums, signing later, activation gates.
Cross-tenant leakage	Tenant-scoped registry resolution and identity context.
File upload risks	Controlled document operations, file metadata validation, storage policy.
Audit failure	Trace events include artifact version, user context, rule results, operation status.

15. AI Application Agent Architecture

The AI Application Agent is a guided design and maintenance assistant. It should not directly generate unrestricted production code. It should generate structured intent and metadata patches, use platform tools, and rely on validators and human approval.

AI Capability	Architecture Boundary
Business conversation	Allowed; must record assumptions and open questions.
Intent generation	Allowed; generates Business Intent Model and Application Intent Model.
Patch generation	Allowed; patch format is preferred over whole-artifact rewrite.
Schema repair	Allowed through validation feedback loop.
Impact analysis	Allowed using registry, dependency graph, operations, and usage search.
Direct publish	Not allowed.
Direct code changes	Not allowed for business application behavior in Phase 0/1.

16. First Vertical Slice: Customer Onboarding Application

Customer Onboarding is the first vertical slice because it demonstrates UI, workflow, data, documents, rules, operations, security, localization, audit, and AI-assisted change without requiring a broad marketplace or visual designer.

Capability	What the Slice Should Prove
Business intent	Application can be described as onboarding capability, roles, rules, documents, workflow.
UI runtime	Dynamic form sections, conditional fields, validation, review screen.
Workflow runtime	Draft, submit, review, approve/reject states.
Rules runtime	Country/customer-type tax and document requirements.
Operation runtime	Save draft, validate tax id, upload document, submit onboarding.
Data runtime	Customer, onboarding case, document metadata persistence abstraction.
Security/policy runtime	Applicant/reviewer/manager/auditor differences.
Observability	Explain why field was visible/required and which operation ran.
AI governance	AI proposes metadata/intent patch and validation catches issues.

17. Recommended Repository and Module Structure

The repository should use the broader product name and separate umbrella platform modules from specific runtime/adapters.

```
appbana-genesis/
  docs/
    adr/
    architecture/
    phase0/
    risk/
```

```
  schemas/
packages/
  business-intent-model/
  application-intent-schema/
  canonical-application-model/
  platform-kernel/
  governance-engine/
  metadata-registry/
  migration-engine/
  conformance-suite/
  ai-application-agent/
  runtime-interaction-ui/
  runtime-workflow/
  runtime-rules/
  runtime-operations/
  runtime-data/
  runtime-integration/
  runtime-security-policy/
  runtime-observability/
  adapter-ui-react/
  adapter-ui-webcomponents/
  adapter-backend-mock/
  adapter-data-memory/
apps/
  customer-onboarding-demo/
examples/
  customer-onboarding/
tools/
  validator-cli/
  trace-viewer/
  migration-runner/
```

18. Implementation Roadmap

Phase	Objective	Exit Evidence
Phase 0: Foundation	Finalize ADRs, application intent model, canonical model, runtime specs, pilot scope.	Approved architecture foundation and executable backlog.
Phase 1: Interaction Runtime Slice	Build Customer Onboarding interaction runtime with React and Tier A Web Components proof.	No page-specific React business logic; deterministic traces.
Phase 2: Full Application Slice	Add workflow, data, semantic operations, security policy, observability hardening.	Customer Onboarding behaves as full application, not just form.
Phase 3: AI Agent Prototype	Natural language to intent/patch, validation loop, preview, approval.	AI patch safely changes app behavior.

Phase 4: Adapter Expansion	Backend/data/deployment adapters and conformance suite.	Technology replaceability proof beyond UI.
Phase 5: Productization	Registry, governance UI, reusable packs, design partners.	80% standard runtime coverage in pilot.

19. Architecture Decision Gates

Gate	Pass Criteria	If Failed
North Star alignment	Architecture supports full app creation, not only UI forms.	Revise architecture before implementation expansion.
Semantic boundary	80% of pilot behavior fits standardized models without custom scripting.	Narrow pilot or improve model rigor.
Runtime determinism	Events produce replayable state/projection/effect traces.	Stop feature expansion and fix kernel.
Adapter proof	Tier A UI proof plus operation/data adapter boundaries work.	Weaken technology-replaceability claim.
Governance proof	Invalid/unsafe intent cannot activate.	No production-like pilot.
AI safety proof	AI patch cannot bypass validation, diff, preview, approval.	AI remains internal helper only.
Economic proof	Change effort is materially lower than conventional baseline.	Do not scale product build.

20. Open Architecture Questions

1. What is the exact boundary between Business Intent Model and Application Intent Model?
2. Which parts should be interpreted at runtime versus generated as code or infrastructure artifacts?
3. How much backend/data adapter generation should be included in the first full vertical slice?
4. What is the minimum viable workflow runtime for Customer Onboarding?
5. What persistence model should be used for the first demo: in-memory, SQLite/Postgres, or mock semantic operations?
6. What conformance evidence is sufficient for UI, operation, and data adapters?
7. How should AppBana Genesis package reusable industry/application templates later?
8. Which name should be final for product and repo: AppBana Genesis is recommended, but branding can still be validated.

21. Immediate Next Actions Before Coding

9. Accept this document as the umbrella architecture baseline.
10. Update existing HLD/LLD/project plan terminology from interaction runtime to AppBana Genesis where appropriate.
11. Create appbana-genesis repository using the recommended module structure.
12. Write ADR-011 through ADR-017 for the broader application-creation architecture.
13. Expand Customer Onboarding from form pilot to full application vertical slice.
14. Define Business Intent Model v0.1 and Application Intent Schema v0.1.
15. Define Canonical Application Model v0.1 with UI/workflow/data/API/security/integration sub-models.
16. Begin implementation with runtime kernel, rules runtime, interaction runtime, semantic operation mock, and trace model.

Appendix A: Relationship to Existing Phase 0 Documents

Existing Document	How It Changes Under AppBana Genesis
Project Plan	Becomes Phase 0/1 execution plan under broader Genesis roadmap.
HLD	Should be revised from interaction runtime HLD to Genesis HLD.
LLD	Should remain valid for interaction runtime vertical slice, but not whole Genesis platform.
Risk Mitigation Plan	Still valid; risks become broader and more important.
ADR Pack	Needs new ADRs for business intent, canonical application model, runtime engines, and technology adapters.
Metadata Schema Spec	Evolves into Application Intent Schema and runtime-specific projection schemas.
Canonical UI Model Spec	Evolves into Canonical Application Model with UI as one sub-model.
Runtime Execution Spec	Remains valid for interaction runtime; additional specs needed for workflow/data/operation runtimes.